2023 Cryptanalysis Contest

Problem 6

Problem statement and team solution

Problem statement

Among post-quantum cryptographic approaches, isogeny-based cryptography relies on the difficulty of computing an isogeny between two elliptic curves E and E' over a finite field. For an isogeny $\phi : E \to E'$, the map ϕ is kept secret. Vélu's formulas allow the isogeny to be computed from $\ker \phi$, so its kernel is also secret. In a typical implementation, $\ker \phi$ is stored instead of the isogeny itself.

The following digital signature scheme is designed using an isogeny problem. Alice has obtained the s component of a signature. Use it to recover the private key and explain why this signature algorithm is insecure. The value of s is provided in the accompanying file `2023pqc_s.txt`.

Setup

- $p = 2^{216}3^{137} - 1$, and $E : y^2 = x^3 + 6x^2 + x$ over $\mathbb{F}_{p^2}$.
- $P_s, Q_s \in E[2^{216}]$ form a generating pair for $E[2^{216}]$; $P_R, Q_R \in E[3^{137}]$ form a generating pair for $E[3^{137}]$.

Key generation

1. Choose the private key $n_s \in \{1, \ldots, 2^{216} - 1\}$ and compute $S = P_s + n_s Q_s$.
2. Compute the isogeny $\phi_s : E \to E_s$ with kernel $\langle S \rangle$.
3. The public key is E_s; the private key is n_s (and the associated isogeny ϕ_s).

Signature generation

Input: a message m. **Output:** (e, s).

1. Choose 256 random points $R_0, \ldots, R_{255} \in E[3^{137}]$.
2. Set $G_i = S + R_i$ and compute $\phi_s(R_i)$ for every i.
3. For each i, compute the isogeny $\beta_i : E_s \to E_i$ with $\ker \beta_i = \langle \phi_s(R_i) \rangle$.
4. Set
$$r = \big(j(E_0) \parallel \cdots \parallel j(E_{255})\big), \qquad e = H(r \parallel m),$$
where j denotes the j-invariant and H is SHA-256.
5. Write $e = (b_0, \ldots, b_{255})$, where each b_i is either 0 or 1.
6. Compute $s = (K_0, \ldots, K_{255})$, where
$$K_i = \begin{cases} G_i, & b_i = 0, \\ \phi_s(R_i), & b_i = 1. \end{cases}$$
7. The signature is (e, s).

Signature verification

Input: (e', s') and a message m. **Output:** valid or invalid.

1. Using $e' = (b'_0, \ldots, b'_{255})$ and $s' = (K'_0, \ldots, K'_{255})$, perform the following computations.
2. If $b'_i = 0$, compute the isogeny $\alpha_i : E \to E'_i$ with kernel $\langle K'_i \rangle$.
3. If $b'_i = 1$, compute the isogeny $\alpha_i : E_s \to E'_i$ with kernel $\langle K'_i \rangle$.
4. Set $r = \big(j(E'_0), \ldots, j(E'_{255})\big)$ and compute $c = H(r \parallel m)$. Output valid if $c = e'$; otherwise, output invalid.

Signature storage

Only the x-coordinate of each elliptic-curve point K_i in the signature s is stored. No special compression function is used. If the x-coordinate is $a + b$i, then a is stored in 448 bits, followed by b in another 448 bits.

For example, suppose the x-coordinates of K_1 and K_2 over $\mathrm{GF}(251^2)$ are $2 + 3$i and $4 + 2$i, respectively, and each component is stored in 8 bits. The stored bytes are `0x02, 0x03, 0x04, 0x02`.

Problem parameters

The following hexadecimal values are the parameters supplied with the problem. Each coordinate is written as its real and imaginary components. Line breaks only wrap a single integer; all leading zeros are retained. The public curve E_s is specified by its Montgomery coefficient A'.

Point	Coord.	Part	Hexadecimal value
P_s	x	Re	00003CCFC5E1F050030363E6920A0F7A4C6C71E63DE63A0E6475AF62 1995705F7C84500CB2BB61E950E19EAB8661D25C4A50ED279646CB48
P_s	x	Im	0001AD1C1CAE7840EDDA6D8A924520F60E573D3B9DFAC6D189941CB2 2326D284A8816CC4249410FE80D68047D823C97D705246F869E3EA50
P_s	y	Re	0001AB066B84949582E3F66688452B9255E72A017C45B148D719D9A6 3CDB7BE6F48C812E33B68161D5AB3A0A36906F04A6A6957E6F4FB2E0
P_s	y	Im	0000FD87F67EA576CE97FF65BF9F4F7688C4C752DCE9F8BD2B36AD66 E04249AAF8337C01E6E4E1A844267BA1A1887B433729E1DD90C7DD2F
Q_s	x	Re	0000C7461738340EFCF09CE388F666EB38F7F3AFD42DC0B664D9F461 F31AA2EDC6B4AB71BD42F4D7C058E13F64B237EF7DDD2ABC0DEB0C6C
Q_s	x	Im	000025DE37157F50D75D320DD0682AB4A67E471586FBC2D31AA32E69 57FA2B2614C4CD40A1E27283EAAF4272AE517847197432E2D61C85F5
Q_s	y	Re	0001D407B70B01E4AEE172EDF491F4EF32144F03F5E054CEF9FDE5A3 5EFA3642A11817905ED0D4F193F31124264924A5F64EFE14B6EC97E5
Q_s	y	Im	0000E7DEC8C32F50A4E735A839DCDB89FE0763A184C525F7B7D0EBC0 E84E9D83E9AC53A572A25D19E1464B509D97272AE761657B4765B3D6
$P_s - Q_s$	x	Re	0000F37AB34BA0CEAD94F43CDC50DE06AD19C67CE4928346E829CB92 580DA84D7C36506A2516696BBE3AEB523AD7172A6D239513C5FD2516
$P_s - Q_s$	x	Im	000196CA2ED06A657E90A73543F3902C208F410895B49CF84CD89BE9 ED6E4EE7E8DF90B05F3FDB8BDFE489D1B3558E987013F9806036C5AC
$P_s - Q_s$	y	Re	00007F65B303A50EF1B4192237611E226A3D13384EF608A6B117365A 16E0EB5112156F2012CB029C819F3330F69BD5C73CCC9A1F1C06CD15
$P_s - Q_s$	y	Im	0000749095AB8A36C841FBF25A5671A67FDE5023131C73F0EC6031C7 E472DAE138FBED0A0BE63C6706CD893EF88D32CC766EC67EC056ED33
P_R	x	Re	00008664865EA7D816F03B31E223C26D406A2C6CD0C3D667466056AA E85895EC37368BFC009DFAFCB3D97E639F65E9E45F46573B0637B7A9
P_R	x	Im	0
P_R	y	Re	00006AE515593E73976091978DFBD70BDA0DD6BCAEEBFDD4FB1E748D DD9ED3FDCF679726C67A3B2CC12B39805B32B612E058A4280764443B
P_R	y	Im	0
Q_R	x	Re	00012E84D7652558E694BF84C1FBDAAF99B83B4266C32EC65B10457B CAF94C63EB063681E8B1E7398C0B241C19B9665FDB9E1406DA3D3846
Q_R	x	Im	0
Q_R	y	Re	0
Q_R	y	Im	0000EBAAA6C731271673BEECE467FD5ED9CC29AB564BDED7BDEAA86D D1E0FDDF399EDCC9B49C829EF53C7D7A35C3A0745D73C424FB4A5FD2
$P_R - Q_R$	x	Re	0001CD28597256D4FFE7E002E87870752A8F8A64A1CC78B5A2122074 783F51B4FDE90E89C48ED91A8F4A0CCBACBFA7F51A89CE518A52B76C
$P_R - Q_R$	x	Im	000147073290D78DD0CC8420B1188187D1A49DBFA24F26AAD46B2D9B B547DBB6F63A760ECB0C2B20BE52FB77BD2776C3D14BCBC404736AE4
$P_R - Q_R$	y	Re	0000DA7A98EA26469B843EBF8D1EE0F00E6786E680AC535F5FF26D25 819549C959497D8E8FB14B1BF6764BD27BAE970D0791AF091E344F22
$P_R - Q_R$	y	Im	000048704FEC03D05B06D8A8197DF08D4946E465099F31B75C63A865 A23CA2AD41A74F05074E9DC3F45C5A26F741A0EA1F3C2E6CDA0BB344
E_s	A'	Re	0000BC39A8C22AFDCAC43EFDD3AB05B45AF0A795D823CD1EC0931D38 6BFDE2D477DFFFBF2C8463460DE0586510E99F24323AB8E54BD0026B
E_s	A'	Im	0000045E901E3BAA12BA1A2D0A37634DEF74A6791039D723962496EB 9C4C368FD50BD06BC7D7EF0B2582ADF73577537BDAA9A064C9AB0DA5

Solution

1 Introduction

The goal of this problem is to recover the private key of an isogeny-based signature algorithm from the supplied signatures. The algorithm operates on elliptic curves over the finite field $\mathbb{F}_{p^2}$, where

$$p = 2^{216}3^{137} - 1.$$

We solved the problem by viewing the abelian group of elliptic-curve points as a module over the integers and exploiting an annihilator of the torsion module given in the problem.

2 Mathematical background

An elliptic curve consists of the points (x, y) satisfying $y^2 = x^3 + ax + b$, together with the point at infinity $\mathcal{O}$. The group law is defined so that three collinear points sum to $\mathcal{O}$; with this addition, the curve forms an abelian group. Since its elements can be added an integer number of times, this group can also be viewed as a $\mathbb{Z}$-module.

An isomorphism between elliptic curves is a bijection preserving this group law. Their j-invariants determine whether the curves are isomorphic over an algebraic closure. For a Montgomery curve

$$M_{A,B}: \quad By^2 = x^3 + Ax^2 + x,$$

the j-invariant is

$$j(M_{A,B}) = \frac{256(A^2 - 3)^3}{A^2 - 4}. \tag{1}$$

An isogeny is a nonconstant surjective homomorphism with a finite kernel. Given a finite subgroup of an elliptic curve, the codomains of isogenies with that kernel are elliptic curves in the same isomorphism class. Thus, the kernel uniquely determines the j-invariant of the image curve. Isogeny-based cryptography relies on the difficulty of recovering the kernel when the source curve and the image curve are known. In the signature algorithm of this problem, a point on the curve is chosen as the private-key point, and the curve obtained from an isogeny whose kernel is the subgroup generated by that point is used as the public key.

3 Attack method

Assuming that the SHA-256 hash of $r \,\|\, m$ behaves randomly, we infer that each K_i equals G_i with probability approximately one half. To identify these values, we multiply the supplied points by 3^{137}, which annihilates R_i. Since $G_i = S + R_i$, this removes R_i and leaves the same value for every such point:

$$3^{137}G_i = 3^{137}S, \qquad 3^{137}R_i = \mathcal{O}.$$

The most frequently occurring values among the $3^{137}K_i$ therefore identify $3^{137}S$, up to sign.

Identifying the repeated point

Only the x-coordinates of the K_i are supplied. We therefore choose either square root

$$y = \pm\sqrt{x^3 + 6x^2 + x}.$$

If this square root does not exist, the value cannot be a G_i and is discarded. The computation produces two frequently recurring points with a common x-coordinate. Writing $X = x_0 + x_1\,\mathrm{i}$, $Y_+ = y_{+,0} + y_{+,1}\,\mathrm{i}$, and $Y_- = y_{-,0} + y_{-,1}\,\mathrm{i}$, their coordinates are as follows. The line breaks below only wrap the hexadecimal integers.

x_0

```
0x12134793ABE4489637E1FCA899164E1155681E9B605D90FC58D50D0C
F017B79F5C809394E954466A4E5A0C4D405603A5811623346F903
```

x_1

```
0xE92FD784F4B7A64C62B7C708430917D1465F67B355B7ED8B46F66F74
A282BAE9E0CDBFC5FFFAA822B22EE446A852E65C7FE2626EEAF1
```

$y_{+,0}$

```
0x2E242178861E9976F86B800735A1154891598C0AE889DF86A5F0D8887
186F2954FF544747F41CC01A3A9B8D93D3B5437792B81093D17
```

$y_{+,1}$

```
0x1F4E5F9C8EDE00DF9F01445BF1BD8C1D0531EE53001A1602C2A33528
6DF849D7F98B6EF1E1F1A968A822753298AB4FB6BFA0707CB1D7B
```

$y_{-,0}$

```
0x205FB059EED25D385676B01BDEAB5667DCB1EA54DC61A1E3AD08A0A77
8E790D6AB00ABB8B80BE33FE5C564726C2C4ABC886D47EF6C2E8
```

$y_{-,1}$

```
0x3F392D4E85645F026FC23C06047DB9F609594C28AD029D954C4790792
07B6280674910E1E0E569757DD8ACD6754B049405F8F834E284
```

The point (X, Y_+) occurs **81 times**, and (X, Y_-) occurs **60 times**. The two points sum to the identity $\mathcal{O}$, so they are negatives of one another and represent the two candidates $\pm 3^{137}S$. We run the recovery procedure below once for each sign; only one candidate yields the private key n_s.

Reducing the problem to a discrete logarithm

The value of P_s is known. Set $P'_s = 3^{137}P_s$ and $Q'_s = 3^{137}Q_s$. We can then compute

$$n_s Q'_s = 3^{137}S - P'_s = K \qquad \text{(a known point).} \tag{2}$$

Since 3^{137} and 2^{216} are coprime, there are integers x, y such that $x3^{137} + y2^{216} = 1$. Hence

$$xP'_s = (x3^{137} + y2^{216})P_s = P_s, \tag{3}$$
$$xQ'_s = (x3^{137} + y2^{216})Q_s = Q_s. \tag{4}$$

Thus P'_s and Q'_s still generate $E[2^{216}]$.

Recovering the nonzero bits

Write the private key in binary as

$$n_s = 2^{i_k} + \cdots + 2^{i_1}, \qquad 216 > i_k > \cdots > i_1 \geq 0. \tag{5}$$

We recover the nonzero bits inductively, starting with the least significant one, i_1. Substituting this expansion into equation (2) gives

$$(2^{i_k - i_1} + \cdots + 1)\, 2^{i_1} Q'_s = K. \tag{6}$$

The point Q'_s has exact order 2^{216}. In particular, we can infer, and directly verify, that

$$2^{215} Q'_s \neq \mathcal{O}, \qquad 2^{216} Q'_s = \mathcal{O}. \tag{7}$$

Consequently,

$$r Q'_s = \mathcal{O}, \quad r \in \mathbb{Z} \quad \Longleftrightarrow \quad 2^{216} \mid r. \tag{8}$$

Algebraically, $\mathbb{Z}$ is a principal ideal domain, so the annihilator ideal $\mathrm{ann}(Q'_s) = \{r \in \mathbb{Z} : r Q'_s = \mathcal{O}\}$ has the form $\langle r \rangle$ for an integer r. Its positive generator is the order of Q'_s. Equation (7) therefore gives

$$2^t K = \mathcal{O} \quad \Longleftrightarrow \quad t \geq 216 - i_1. \tag{9}$$

By testing successive powers of 2 and locating the transition to zero, we can recover $i_1, \ldots, i_k$ in order. Figure 1 shows the procedure, with $t = 0$ explicitly included in the loop.

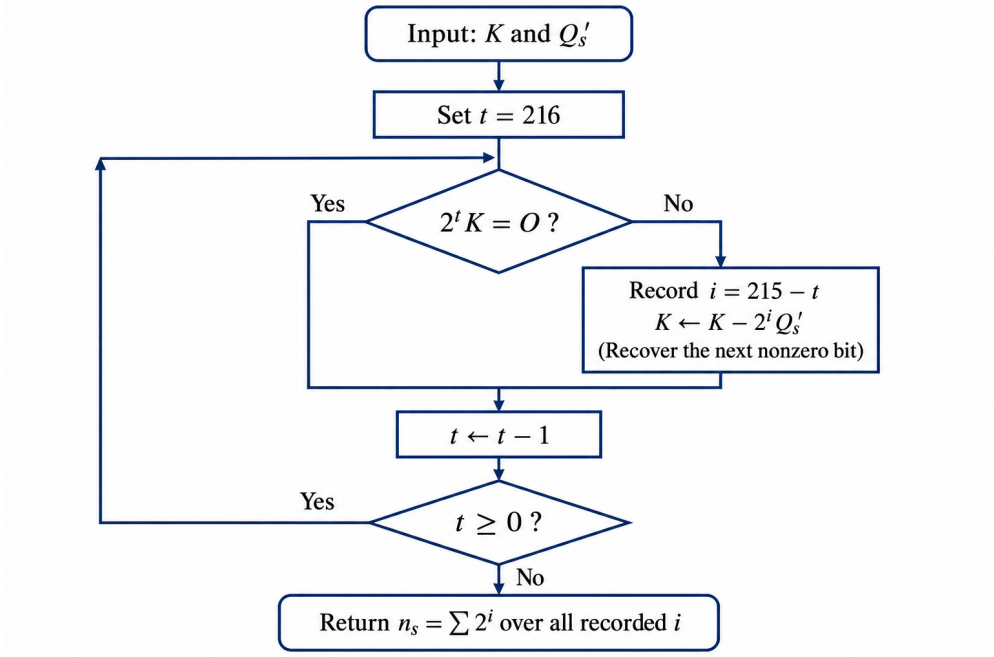


Figure 1: Private-key recovery from $K = n_s Q'_s$. Here $\mathcal{O}$ is the point at infinity.

We used SageMath for the attack. The code is saved as 호모알로라_6번.sage.

4 Conclusion

The attack described above recovered the private key

```
3704392516436702187065769895618952386525860013523846760016170096 4
```

To verify it, we computed $S = P_s + n_s Q_s$ using the recovered n_s, constructed an isogeny with kernel $\langle S \rangle$, and confirmed that its codomain has the same j-invariant as E_s.

The signature algorithm is intended to be secure because of the following obstacles:

1. Given the signature $s = \{K_i\}$, it is difficult to determine which values are G_i and which are $\phi(R_i)$.
2. Even after identifying a value $K_i = S + R_i$, the randomness of R_i appears to prevent recovery of S.
3. Even if S is known, recovering n_s from $S = P_s + n_s Q_s$ is a discrete logarithm problem and is therefore expected to be infeasible in polynomial time.

As described in Section 3, we overcame these obstacles as follows:

1. The behavior of SHA-256 suggests that the two alternatives G_i and $\phi(R_i)$ occur with similar probabilities. We test whether a supplied K_i lies on $y^2 = x^3 + 6x^2 + x$; values that do are treated as likely candidates for G_i.
2. The R_i have order 3^{137}. With sufficiently many G_i candidates, we can therefore identify the repeated value $3^{137} K_i = 3^{137} S$.
3. From $3^{137} S = 3^{137} P_s + n_s 3^{137} Q_s$, we obtain $n_s 3^{137} Q_s = 3^{137} S - 3^{137} P_s$. We then exploit the order 2^{216} of Q_s to recover n_s by a divide-and-conquer procedure, with the original reporting a complexity of $O(n \log n)$.

The original concludes that this attack shows the proposed construction does not function as a zero-knowledge proof (ZKP) and therefore does not provide sufficient security as a signature algorithm.